

2. НАСТРОЙКА РАБОЧЕГО ОКРУЖЕНИЯ

В ходе прохождения учебной практики использовался язык программирования Python, среда разработки (IDE) PyCharm, который представлен на рисунке1.



Рисунок 1 – Иконка PyCharm

Python – это высокоуровневый, интерпретируемый язык программирования общего назначения. Широко используется для разработки программ, веб-сайтов, игр, анализа данных, искусственного интеллекта и автоматизации задач [1].

Основные особенности Python:

- простой и понятный синтаксис
- код читается почти как обычный текст
- поддерживает объектно-ориентированное, процедурное и функциональное программирование
- работает на разных операционных системах

Достоинства Python:

- Простота изучения
- Большое количество библиотек

Недостатки Python:

- Низкая скорость работы
- Большое потребление памяти

Python подходит для разработки приложения кредитного калькулятора благодаря тому, что имеет простой и понятный синтаксис, удобные средства для математических вычислений и позволяет быстро создать приложение с графическим интерфейсом. Кроме того, Python выделяется среди других языков программирования снижением объёма кода, необходимого для реализации вычислений, отсутствием необходимости в компиляции и сложной структурной организации программы. Подключение библиотек выполняется легко и эффективно, что ускоряет процесс разработки, а возможность создания настольных приложений с графическим интерфейсом делает его оптимальным.

IDE PyCharm

PyCharm – это интегрированная среда разработки (IDE) для языка Python. Она предназначена для удобного написания, запуска и отладки программ. Среда разработки представлена на рисунке 2 [2].

Плюсы PyCharm:

- Среда разработки разработана специально для работы с этим языком и предоставляет широкий спектр функций, нацеленных на разработку на Python.

- Интегрированный отладчик.
- Доступна бесплатная и открытая версия для сообщества.
- Есть версии для всех основных ОС (Windows, Linux, MacOS).

Некоторые минусы PyCharm:

- Сложности в настройке.
- Существует бесплатная Community Edition, которая имеет ограниченный функционал по сравнению с платной версией.

- Ограниченная поддержка других языков.

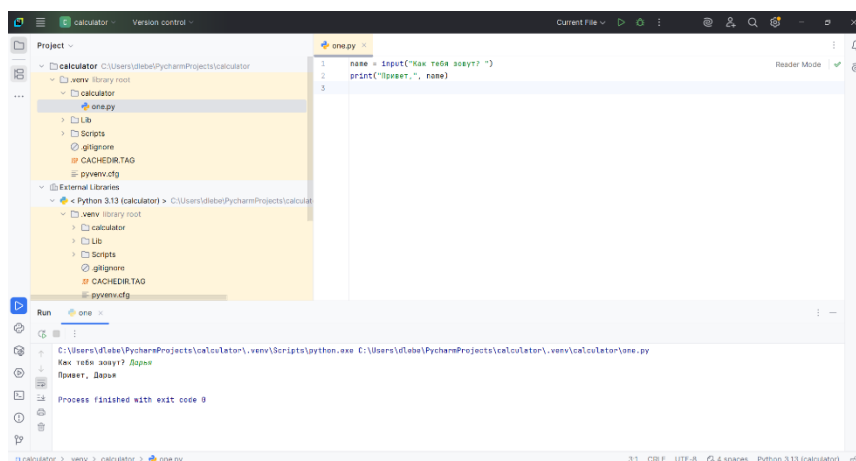


Рисунок 2 – Среда разработки

Для системы контроля версий в соответствии с заданием использована система GitHub. Профиль репозитория представлен на рисунке 3

Ссылка на профиль: <https://github.com/darll3>

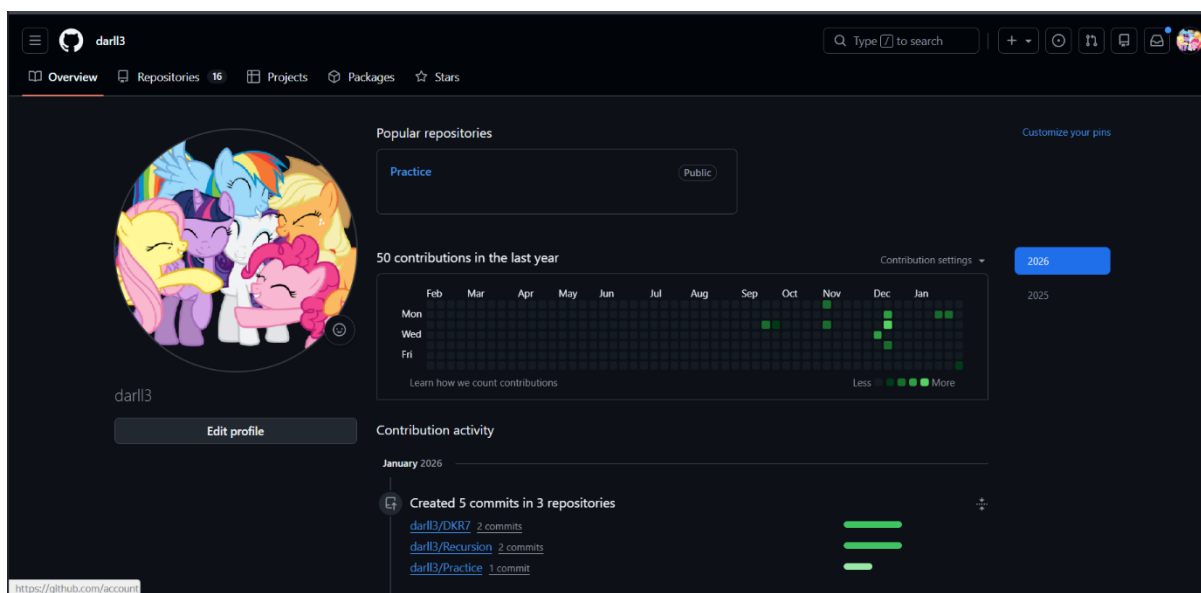


Рисунок 3 – Профиль

Ссылка на репозиторий: <https://github.com/darll3/Practice>. Репозиторий представлен на рисунке 4.

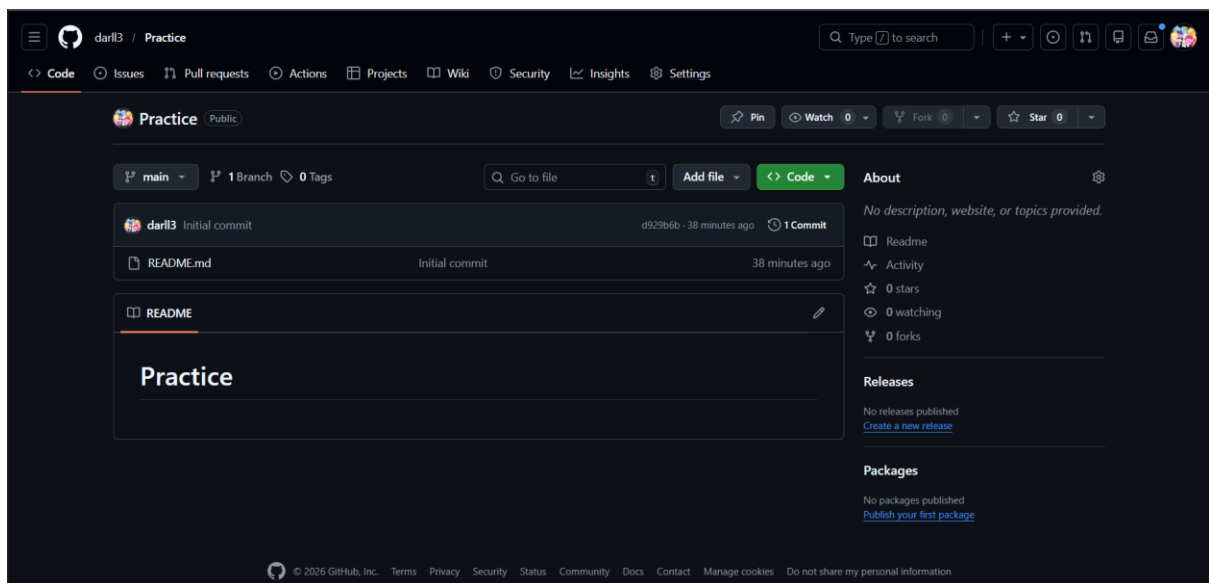


Рисунок 4 – Репозиторий

На текущем этапе заложен следующий технологический стек:

Библиотека	Назначение	Обоснование выбора
PyQt6	Графический интерфейс (GUI)	Мощный фреймворк с огромным количеством виджетов. Поддерживает сложную стилизацию и обеспечивает нативный вид окон в системе.
Datetime	Работа с датами	Необходима для корректного расчета временных периодов, дедлайнов или графиков.

4 ОПИСАНИЕ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

4.1 Анализ предметной области

Кредит – это форма финансовых отношений, основанная на доверии, при которой одна сторона (кредитор) передаёт другой (заёмщику) денежные средства на условиях обязательного возврата в определённый срок и с уплатой процентов. Если деньги выдаёт банк, эти отношения фиксируются кредитным договором.

Как работает кредит и зачем он нужен?

Механизм кредитования:

1. Клиент выбирает подходящую программу и обращается в банк с заявкой.
2. Банк выдаёт необходимую сумму на заранее согласованных условиях.
3. Заёмщик получает средства и обязуется вернуть их, но не сразу, а в несколько ежемесячных платежей. Каждый платёж состоит из двух частей: проценты за пользование и часть основного долга. Проценты – это плата за доступ к чужим деньгам. Чем дольше срок и выше сумма, тем больше общая переплата. Ставка зависит от многих факторов: уровня ключевой ставки ЦБ, надёжности клиента, наличия залога или поручителя.
4. Условия фиксируются в договоре. Деньги поступают на счёт клиента или направляются продавцу, если кредит целевой [3].

Основные параметры кредита:

Сумма кредита – это сумма, которую вы получили от банка и теперь должны вернуть по договору кредитования.

Основной долг – это часть суммы кредита, которую осталось вернуть банку для полного погашения.

Ставка по кредиту – это параметр для расчета суммы, которую нужно заплатить банку за использование кредита помимо возврата долга. Поэтому чем выше ставка, тем дороже обходится ваш кредит.

Проценты – так называют сумму, которую платят за использование кредита. Ее рассчитывают и начисляют раз в сутки, а погашение происходит, когда вы вносите платежи по кредиту. До этого момента проценты просто копятся.

Сумма, которую начислят в качестве процентов, зависит от ставки и остатка долга по кредиту. Чем больше эти параметры, тем больше сумма процентов накапливается каждый день.

Срок кредита – это отрезок времени, за который предстоит полностью погасить кредит.

Выбирая срок, нужно помнить, что чем дольше вы платите кредит, тем меньше будет обязательный платеж. При этом чем дольше срок, тем больше вы заплатите банку помимо возврата суммы, которую взяли в кредит.

Регулярный платеж – это сумма, которую требуется вносить каждый месяц по договору кредита, пока он не будет погашен. Иногда регулярный платеж называют ежемесячным или обязательным.

Прежде чем платеж пойдет на погашение вашего долга, из него вычитают проценты и стоимость дополнительных услуг, если такие подключены.

Дополнительные услуги – это сервисы, которые влияют на процесс погашения. Дополнительные услуги бывают обязательными и необязательными. Разница в том, что если отключить обязательные услуги, то изменятся условия кредита: например, банк будет вынужден увеличить

ставку по кредиту, а подключение или отключение необязательных услуг никак не влияет на условия кредита.

Расчетный период – это промежуток между двумя ближайшими платежами. Он влияет на сумму процентов в платежах: чем дольше период, тем больше накопится процентов к дате погашения.

Учет дней в расчетном периоде начинается на следующие сутки после платежа и заканчивается в дату нового погашения.

Полная стоимость кредита – это плата банку за использование кредита. Она состоит из процентов и стоимости обязательных услуг [4].

Есть несколько видов кредитования, и каждый из них предназначен для разных целей:

- Потребительский. Его обычно берут для того, чтобы купить товары и услуги. Вы можете тратить эти деньги на любые цели: крупные покупки, оплату счетов, отпуск и другие.

- Ипотечный. Его берут специально для покупки недвижимости. Обычно сумма кредита довольно большая, поэтому и срок может достигать до 30 лет.

- Автокредит. Подойдет, чтобы купить автомобиль. Обычно его сумма и срок меньше, чем у ипотечного кредита.

- Кредит на образование. С его помощью можно оплатить обучение в любом аккредитованном российском вузе или колледже по основным образовательным программам.

- Рефинансирование кредитов. Это способ закрыть кредиты в других банках и получить более комфортные условия.

Что нужно чтобы получить кредит?

Для разных кредитов банки выдвигают разные требования к заемщику.

– Возраст: чтобы взять кредит, вы должны быть старше 18 лет. Это связано с тем, что по закону несовершеннолетние не могут заключать юридически обязывающие сделки.

– Доход: банк хочет быть уверен, что вы сможете вернуть деньги, поэтому вы должны иметь регулярный источник дохода.

– Кредитная история: банк проверяет вашу кредитную историю, чтобы узнать, как вы ранее выплачивали кредиты [5].

Калькулятор – электронное вычислительное устройство для выполнения операций над числами или алгебраическими формулами.

Калькулятор заменил механические вычислительные устройства, такие, как счёты, логарифмические линейки, механические или электромеханические арифмометры, а также математические таблицы.

В зависимости от сферы применения калькуляторы делятся на простейшие, бухгалтерские, инженерные (научные), финансовые. В отдельные классы обычно выделяют программируемые калькуляторы, дающие возможность выполнения сложных вычислений по предварительно заложенной программе, а также графические – поддерживающие построение и отображение графиков.

Влияние IT-индустрии на развитие калькуляторов.

Исторически для подобных вычислений использовались механические устройства (арифмометры, логарифмические линейки) и таблицы, а затем – электронные калькуляторы. Развитие IT-индустрии, в частности миниатюризация и рост вычислительной мощности, привело к появлению специализированных финансовых и программируемых калькуляторов, способных выполнять сложные формулы [6].

Что такое кредитный калькулятор?

Это онлайн-инструмент, который позволяет сделать предварительный расчет по кредиту: ежемесячный платеж, общую сумму выплат и переплату. Пользователь указывает сумму, срок кредита, ставку, тип платежей – и сразу получает результат [7].

4.1.1 Анализ проблемы

При оформлении кредита заёмщик сталкивается с необходимостью оценить свои финансовые возможности и понять, насколько выбранные условия будут для него выгодны. Однако на практике сделать это без специальных инструментов бывает сложно. Расчёт ежемесячных платежей, общей суммы выплат и переплаты по кредиту вручную требует знания формул, внимательности и учёта множества параметров, таких как процентная ставка, срок кредита, сумма основного долга и тип платежей.

Многие потенциальные заёмщики не обладают достаточными финансовыми знаниями, что может привести к неправильной оценке своих возможностей, выбору слишком большого срока или суммы кредита, а также к недооценке итоговой переплаты. В результате это может вызвать финансовые трудности, просрочки платежей и ухудшение кредитной истории.

Кроме того, условия кредитования в разных банках отличаются, и для их сравнения необходимо выполнять несколько расчётов с разными входными данными. Без автоматизированного инструмента этот процесс становится долгим и неудобным, а вероятность ошибок возрастает.

Таким образом, проблема заключается в отсутствии у пользователей простого и наглядного способа быстрого и точного расчёта параметров кредита. Решением данной проблемы является разработка кредитного калькулятора, который позволит автоматизировать вычисления, снизить и помочь пользователю принять более обоснованное финансовое решение.

4.1.2 Обзор аналогов

Первым рассматриваемым аналогом, который представлен на рисунке 5, является «КонсультантПлюс» – это онлайн-инструмент, доступный на сайте системы, который помогает пользователю рассчитать параметры кредита: ежемесячные платежи, общую сумму выплат, переплату и итоговый график выплат при различных вариантах платежей и условиях досрочного погашения.

Преимущества кредитного калькулятора «КонсультантПлюс»:

1. Удобный автоматический расчёт:
2. Поддержка разных параметров кредита. Можно учитывать изменение процентной ставки в процессе, варианты досрочных погашений, перенос платежей, что повышает точность расчёта.
3. Возможность анализа разных сценариев. Пользователь может быстро пересчитать результаты при изменении условий, что полезно для выбора оптимального кредита.

Недостатки кредитного калькулятора «КонсультантПлюс»

1. Ограниченный функционал по сравнению со специализированными финансовыми приложениями. Он не предлагает дополнительные финансовые анализы (например, сравнение нескольких предложений банков одновременно или визуальные графики).
2. Интерфейс может быть сложен для новичков.
3. Требуется интернет-доступа и некоторой подготовки. Чтобы правильно использовать все функции, пользователь должен понимать финансовые термины и условия кредита.

Сумма кредита

Дата выдачи

Срок кредита

Ставка кредита

Вид платежа ?

Дата платежей ?

Первый платёж

Рассчитать

Рисунок 5 – КонсультантПлюс

Следующим рассматриваемым аналогом, представленным на рисунке 6 станет calcus.ru – это инструмент для расчёта параметров кредита с учётом возможности внести досрочное погашение, изменения процентной ставки, выбора даты и типа платежей.

Преимущества кредитного калькулятора calcus.ru:

1. Калькулятор учитывает: дату выдачи кредита, изменение процентной ставки, выбор дня для ежемесячного платежа и возможность внесения досрочных платежей.
2. Учёт досрочных погашений
3. Можно вводить как разовые, так и регулярные досрочные платежи, а сервис корректирует график выплат.
4. Разные типы платежей

5. Пользователь сам выбирает, будет ли ежемесячный платёж аннуитетным или дифференцированным, что влияет на переплату.

Недостатки кредитного калькулятора calcus.ru

1. Сложность для новичков
2. Множество параметров и настроек может запутать пользователя, который впервые сталкивается с кредитными расчётами.
3. Нужно понимать финансовые термины. Чтобы использовать весь функционал правильно.
4. Требуется интернет-доступа и загрузки страницы

Кредитный калькулятор с досрочным погашением

Сумма кредита		₽ ▾
Срок кредита		лет ▾
Дата выдачи кредита	24.01.2026	📅
Процентная ставка		%
	+ Добавить ставку	
Тип ежемесячных платежей	☒ Аннуитетные ⓘ ☐ Дифференцированные ⓘ	
Ежемесячные платежи	в день выдачи кредита ▾	
	☐ Первый платеж - только проценты	☐ Округлять платеж
	☐ Пропустить первый месяц	☐ Переносить платеж с выходных дней
	☐ Увеличить ЕП до размера процентов ⓘ	
	+ День платежа изменился	
Досрочные погашения	+ Добавить досрочный платеж	
Общий ежемесячный платеж ⓘ	+ Добавить общий ежемесячный платеж	
Кредитные каникулы ⓘ	+ Добавить кредитные каникулы	

Рисунок 6 – Калькулятор calcus.ru

Аналог, который представлен на рисунке 7, является Banki.ru – это бесплатный онлайн-инструмент, с помощью которого можно быстро рассчитать ежемесячный платёж по кредиту, общую сумму выплат и переплату, указав сумму кредита, срок, процентную ставку и тип платежей (аннуитетные или дифференцированные).

Преимущества кредитного калькулятора «Banki.ru»

1. Быстрый и точный расчёт.
2. Поддержка разных типов расчётов. Сервис позволяет выбирать между аннуитетными и дифференцированными платежами.
3. Бесплатный доступ и простота. Инструмент доступен бесплатно и не требует регистрации – подходит для предварительного анализа перед обращением в банк.

Недостатки кредитного калькулятора «Banki.ru»

1. Ограничение по детализации условий. Калькулятор рассчитывает базовые параметры, но не учитывает дополнительные банковские условия, такие как комиссии, страхование и т.п.
2. Зависимость от корректности введённых данных. Результаты расчётов напрямую зависят от того, насколько точно пользователь ввёл параметры, и при ошибках итог может быть неточен.

Результаты расчета	
Ежемесячный платеж	7295 ₽
Сумма кредита	300 000 ₽
Переплата	137 700 ₽
Общая сумма займа	437 700 ₽

Рисунок 7 – Калькулятор Banki.ru

Последний рассматриваемый аналог, представленный на рисунке 8 размещен на портале Сравни – онлайн- инструмент, который помогает рассчитать условия кредита: ежемесячный платёж, общую сумму выплат, переплату и график погашений.

Преимущества кредитного калькулятора «Сравни»

1. Полный расчёт основных параметров кредита
2. Возможность выбора типа платежей
3. Быстрый онлайн-доступ и простота использования

Недостатки кредитного калькулятора «Сравни»:

1. Зависимость от введённых данных. Как и у большинства калькуляторов, точность расчёта напрямую зависит от правильности введённых параметров. Ошибки в данных могут привести к неверным выводам о выплатах.
2. Не учитывает дополнительные банковские условия

The screenshot displays the 'Сравни' credit calculator interface. On the left, there are input fields for: 'Вариант расчета' (set to 'Расчет ежемесячного платежа'), 'Сумма кредита' (500 000 Р), 'Срок кредита' (5 лет), 'Ставка до' (30 %), and 'Начало выплат' (24.01.2026). Below these is a section for 'Тип платежей' with two buttons: 'Аннуитетный' and 'Дифференцированный'. On the right, a summary box shows: 'Ежемесячный платеж' (16 176,69 Р), 'Сумма кредита' (500 000,00 Р), 'Переплата' (470 601,77 Р), and 'Общая сумма кредита' (970 601,99 Р). At the bottom right, there is a 'Подбор кредита' section with a description, a 'Перейти' button, and an icon of a bank building with a checkmark.

Рисунок 8 – Калькулятор «Сравни»

Вывод

Таким образом, общими недостатками всех рассмотренных кредитных калькуляторов являются то, что расчёты носят примерный характер и могут отличаться от реальных условий банка. Для получения корректных результатов пользователю необходимо правильно вводить данные и понимать основные условия кредита, что может быть сложно для неподготовленного человека. Кроме того, калькуляторы не учитывают все дополнительные услуги и комиссии, которые могут влиять на итоговую сумму выплат.

4.2 Разработка структуры приложения и алгоритмов функционирования

После запуска программы происходит инициализация интерфейса загружается главное окно со всеми полями ввода и элементами управления. Далее пользователь приступает к вводу основных данных: указывает сумму кредита, дату выдачи и срок кредита. Затем следует блок выбора дополнительных параметров, где пользователь настраивает вид платежа, дату платежей, ставку кредита, тип первого платежа, комиссии, а также параметры досрочного погашения. Предусмотрена кнопка сброса настроек для возврата всех полей к значениям по умолчанию. После заполнения всех данных пользователь нажимает кнопку «Расчет». При необходимости пользователь может изменить параметры и выполнить расчет заново или нажать кнопку выхода для завершения программы. Схема алгоритма представлена на рисунке 9.

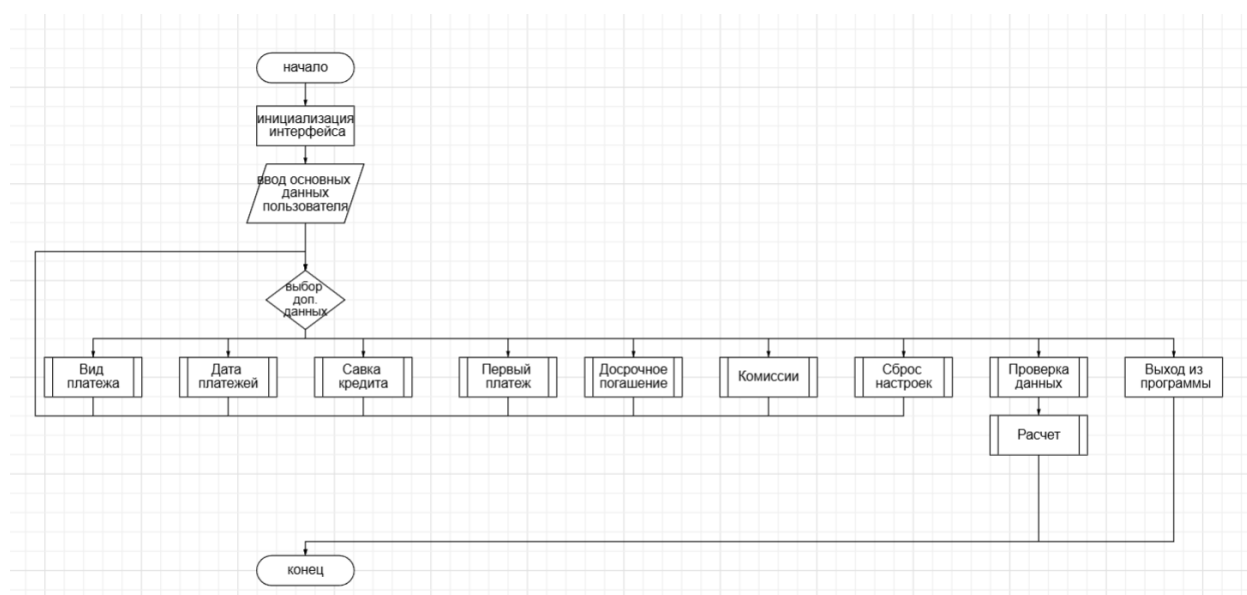


Рисунок 9 – Схема алгоритма

Приводится прототип интерфейса, нарисованный в Figma, который представлен на рисунке 10.

Параметры кредита

Сумма кредита: 100000

Дата выдачи: 04.04.2026

Срок кредита: 12 мес.

Ставка кредита: 12

Фиксированная / Изменяемая

Вид платежа: Аннуитетный / Дифференцированный

День платежей: В день выдачи кредита / В последний день месяца / Свой день

Первый платеж: Только проценты / Проценты и часть основного долга

Результат:

Ежемесячный платеж: 1000.00 руб.

Общая сумма платежа: 107099.51 руб.

Переплата: 7099.51 руб.

График платежей

Дата	Платеж	Ост. долг	Проценты	Остаток
04.05.2026	1 000.00	0.00	1 000.00	100 000.00
04.06.2026	9 645.41	8 645.41	1 000.00	91 354.59
04.07.2026	9 645.41	8 731.86	913.55	82 622.73
04.08.2026	9 645.41	8 819.18	826.23	73 803.55
04.09.2026	9 645.41	8 907.37	738.04	64 896.18
04.10.2026	9 645.41	8 996.45	648.96	55 899.73
04.11.2026	9 645.41	9 086.41	559.00	46 813.32
04.12.2026	9 645.41	9 177.27	468.13	37 636.05
04.01.2027	9 645.41	9 269.05	376.36	28 367.00
04.02.2027	9 645.41	9 361.74	283.67	19 005.26
04.03.2027	9 645.41	9 455.35	190.05	9 549.91
04.04.2027	9 645.41	9 549.91	95.50	0.00

Досрочное погашение

Сумма досрочного платежа: 0.00

Тип погашения: Снизить платеж / Снизить срок

Частота: Единовременно / Раз в месяц / Раз в год

Дата: - -

Комиссии

Единовременная / Ежемесячная

Введите значение: руб.

Рассчитать / Очистить

Рисунок 10 – Интерфейс

В данном разделе рассмотрены алгоритмы работы системы. Рассмотрен алгоритм ввода данных, алгоритм расчета платежей и алгоритм обработки досрочных погашений.

Приводится прототип интерфейса, нарисованный в Figma. Интерфейс включает меню выбора типа кредита, панель ввода параметров кредита, кнопки управления и область вывода результатов.

4.3 Разработка алгоритмов работы программы и программного обеспечения

В данном разделе будут произведены разработки алгоритмов функционирования программы, разработан интерфейс программы, реализованы основные функции программы.

4.3.1 Разработка программного обеспечения

Программное обеспечение разработано в среде PyCharm на языке Python.

Программное обеспечение состоит из двух основных классов. Класс Program отвечает за создание графического интерфейса приложения и реализует все элементы взаимодействия с пользователем, включая поля ввода, кнопки, таблицу и отображение результатов. В нем содержатся функции инициализации интерфейса (`init_ui`), применения стилей (`apply_styles`), управления состоянием элементов (`toggle_sections`, `on_rate_type_changed`), очистки данных (`clear_all_data`, `clear_all_errors`), а также вспомогательные методы для валидации и обработки пользовательского ввода.

Исходный код программного обеспечения представлен в приложении А.

Функция `init_ui()` относится к классу Program и предназначена для создания и инициализации графического интерфейса пользователя. Внутри функции формируется основное окно приложения, создаются и настраиваются все элементы интерфейса: поля ввода, кнопки, выпадающие списки, таблица и блоки отображения результатов. Фрагмент кода представлен на рисунке 11.

```

def init_ui(self):
    self.setWindowTitle("Калькулятор")
    #добавляем скролл
    self.main_scroll = QScrollArea()
    # содержимое подстраивается под размер
    self.main_scroll.setWidgetResizable(True)
    # минус лишняя рамка вокруг скролла
    self.main_scroll.setFrameShape(QFrame.Shape.NoFrame)
    # делаем его главным элементом окна
    self.setCentralWidget(self.main_scroll)
    # создаем виджет, как подложку под все остальное и добавляем ее внутрь скролла
    self.scroll_content = QWidget()
    self.main_scroll.setWidget(self.scroll_content)
    # создаем горизонтальную макет и делаем там отступы
    self.main_layout = QHBoxLayout(self.scroll_content)
    self.main_layout.setContentsMargins(20, 20, 20, 20)
    # отступы между левой и правой панелью
    self.main_layout.setSpacing(20)

```

Рисунок 11 – Функция init_ui()

Функция apply_styles() относится к классу Program и предназначена для настройки внешнего вида графического интерфейса приложения. Функция содержит настройки цвета фона, границ, шрифтов и размеров для различных компонентов, таких как окна, кнопки, поля ввода, таблицы и всплывающие подсказки. Фрагмент кода представлен на рисунке 12.

```

def apply_styles(self):
    self.setStyleSheet("""
        #MainFrame, #resultFrame, #tabelname {
            background-color: #E2F9E8;
            border-radius: 20px;
            border: 2.5px solid #74c43f;
        }

        #Header, #tabelheader, #Result {
            font-size: 18px;
            font-weight: bold;
            color: #333;
        }

        #calculateBTN {
            background: #90f59a;
            min-height: 40px;
            font-weight: bold;
            border-radius: 10px;
        }

        #clearBTN {
            background: #f66565;
            color: white;
            min-height: 40px;
            font-weight: bold;
        }
    """)

```

Рисунок 12 – Функция apply_styles()

Функция `toggle_sections()`. Внутри функции выполняется проверка состояния соответствующих чекбоксов, после чего включаются или отключаются группы элементов ввода. Фрагмент кода представлен на рисунке 13.

```
# чекбоксы
def toggle_sections(self):
    # досрочка, проверка на то, стоит ли галочка
    is_early = self.check_early.isChecked()
    # создаем список всех полей которые находятся в досрочке
    early_widgets = [
        self.early_sum_input, self.btn_reduce_sum, self.btn_reduce_term,
        self.btn_once, self.btn_monthly, self.btn_yearly, self.early_date_input
    ]
    # проходим по списку элементов
    for w in early_widgets:
        # включаем или выключаем элемент
        w.setEnabled(is_early)
        # если галочки нет, то убираем красную рамку
        if not is_early:
            self.clear_field_error(w)

    # чекбокс у комиссии
    is_comm = self.check_comm.isChecked()
    comm_widgets = [
        self.btn_comm_once, self.btn_comm_month,
        self.comm_val_input, self.comm_unit_combo
    ]
    for w in comm_widgets:
        w.setEnabled(is_comm)
        if not is_comm:
            self.clear_field_error(w)
```

Рисунок 13 – Функция `toggle_sections()`

Функция `clear_all_data()`. Внутри функции выполняется очистка всех полей ввода, удаление результатов расчета, сброс таблицы платежей к начальному виду, а также снятие всех выбранных параметров (чекбоксов и кнопок). Фрагмент кода представлен на рисунке 14.

```
# кнопка очистить
def clear_all_data(self):
    # проходим по списку всех текстовых полей
    for field in self.all_inputs:
        field.clear()

    # снимаем красные поля
    self.clear_all_errors()
    # и делаем таблицу по дефолту
    self.clear_table_to_default()

    # очистка полей в части результат
    self.res_payment.clear()
    self.res_total.clear()
    self.res_overpay.clear()

    # сброс чекбоксов
    self.check_early.setChecked(False)
    self.check_comm.setChecked(False)
```

Рисунок 14 – Функция `clear_all_data()`

Функция `clear_all_errors()`. Внутри функции формируется список основных полей, после чего вызывается механизм снятия подсветки ошибок, что приводит к возврату стандартного внешнего вида элементов. Исходный код представлен на рисунке 15.

```
# очищаем ошибки
def clear_all_errors(self):
    # создаем список с основными полями
    all_to_check = [self.sum_input, self.term_input, self.rate_input,
                    self.rate_input_fixed, self.date_input, self.custom_day_input]
    # передается пустой список для того чтобы убрать красную обводку
    self.highlight_errors([])
```

Рисунок 15 - Функция `clear_all_errors()`

Функция `validate_input_start()` относится к классу `Program` и предназначена для проверки корректности ввода числовых данных. Внутри функции анализируется первый символ введенного текста, и если он является точкой или запятой, данный символ автоматически удаляется. Дополнительно пользователю отображается всплывающее сообщение об ошибке. Исходный код представлен на рисунке 16.

```
# функция которая запрещает начинать ввод с точки или запятой
def validate_input_start(self, widget):
    text = widget.text()
    if text.startswith('.') or text.startswith(','):
        # убирает точку и запятую
        widget.setText(text.lstrip('.,'))
        #сообщение об ошибке
        QToolTip.showText(
            widget.mapToGlobal(QPoint(0, widget.height())) ,
            "Число не может начинаться с разделителя",
            widget
        )
```

Рисунок 16 – Функция `validate_input_start()`

Функция `create_help_button()` предназначена для создания кнопки с подсказкой. Внутри функции создается кнопка со знаком вопроса, настраиваются её размеры и внешний вид, а также добавляется обработчик события нажатия. При нажатии на кнопку отображается всплывающая подсказка с переданным текстом. Исходный код представлен на рисунке 17.

```
# создание кнопки со знаком вопроса
def create_help_button(self, text):
    btn = QPushButton("?")
    btn.setFixedSize(22, 22)
    btn.setCursor(Qt.CursorShape.PointingHandCursor)
    btn.setObjectName("helpButton")
    # убираем всплывающую подсказку (она появляется если долго держать курсор на кнопке)
    btn.setToolTip("")
    # а вот тут создаем подсказку по клику
    btn.clicked.connect(lambda: QToolTip.showText(
        btn.mapToGlobal(QPoint(25, 0)),
        text,
        btn
    ))

    return btn
```

Рисунок 17 – Функция `create_help_button()`

Функция `clear_table_to_default()` предназначена для инициализации таблицы значениями по умолчанию. Внутри функции задается фиксированное количество строк, после чего каждая ячейка таблицы заполняется пустым значением и окрашивается в белый цвет. Это позволяет визуально отобразить структуру таблицы до выполнения расчетов. Исходный код представлен на рисунке 18.

```
# делает сразу 10 пустых строк, для обозначения что это таблица
def clear_table_to_default(self):
    self.payment_tabel.setRowCount(10)
    # проходимся по строкам и столбцам
    for i in range(10):
        for j in range(self.payment_tabel.columnCount()):
            # создаем объект, те ячейку и записываем в нее пустую строку
            item = QTableWidgetItem("")
            item.setBackground(QColor("white"))
            # созданную ячейку кладем в таблицу на пересечение строки и столбца
            self.payment_tabel.setItem(i, j, item)
```

Рисунок 8 - Функция `clear_table_to_default()`

Функция `add_rate_row()`. Внутри функции создается контейнер, содержащий поля для ввода даты и значения ставки, а также кнопку удаления строки. Для полей ввода настраиваются ограничения и обработчики событий, обеспечивающие корректность ввода и снятие ошибок. При нажатии на кнопку удаления соответствующая строка удаляется из интерфейса. Созданный элемент добавляется в общий контейнер для хранения изменений ставок. Фрагмент кода представлен на рисунке 19.

```
# для изменения ставки
def add_rate_row(self):
    row_widget = QWidget()
    row_layout = QHBoxLayout(row_widget)
    row_layout.setContentsMargins(15, 10, 15, 10)

    date_edit = QLineEdit()
    date_edit.setPlaceholderText("ДД.ММ.ГГГГ")
    date_edit.setInputMask("99.99.9999")
    # убираем красную рамку если пользователь начал что-то писать
    date_edit.textChanged.connect(lambda: self.clear_field_error(date_edit))

    rate_edit = QLineEdit()
    rate_edit.setPlaceholderText("0.00%")
    # текст по центру поля, разрешаем писать только числа
    rate_edit.setAlignment(Qt.AlignmentFlag.AlignCenter)
    rate_edit.setValidator(self.numeric_validator)
```

Рисунок 19 - Функция `add_rate_row()`

Функция `on_rate_type_changed()`. Внутри функции проверяется, выбрана ли изменяемая ставка, после чего соответствующие элементы интерфейса либо отображаются, либо скрываются. При возврате к фиксированной ставке выполняется удаление ранее добавленных строк с изменяемыми значениями. Фрагмент кода представлен на рисунке 20.

```

#кнопка добавить только при изменяемой ставке
def on_rate_type_changed(self):
    # нажата ли сейчас кнопка изменяемая
    is_var = self.btn_variable.isChecked()
    # если да, то показываем контейнер с датами и ставкой и кнопку добавить
    self.rate_container.setVisible(is_var)
    self.btn_add_rate.setVisible(is_var)
    # поле для фиксированной показываем если не нажата изменяемая ставка
    self.rate_input_fixed.setVisible(not is_var)
    # если вернулся к фиксированной ставке
    if not is_var:
        #очищает строки и возвращает фиксированную ставку
        # идем с конца списка,удаляем все кроме первой строк
        for i in reversed(range(1, self.rate_changes_layout.count())):
            # берем строку под номером i и если она нашлась то удаляем
            widget = self.rate_changes_layout.itemAt(i).widget()
            if widget: widget.deleteLater()

```

Рисунок 20 – Функция on_rate_type_changed()

Функция highlight_errors() предназначена для визуального выделения полей с ошибками ввода. Внутри функции сначала происходит сброс состояния всех полей, после чего для элементов, содержащих ошибки, устанавливается специальное свойство, изменяющее их внешний вид (например, добавление красной рамки). Исходный код представлен на рисунке 21.

```

def highlight_errors(self, widgets):
    #проходим по всем полям ввода
    for field in self.all_inputs:
        # сначала убираем ошибку, после заставляем забыть как выглядило поле и заново делаем проверку
        field.setProperty( name: "error", value: "false")
        field.style().unpolish(field)
        field.style().polish(field)

    #проходим по полям в которых есть ошибка
    for widget in widgets:
        # ошибка-да, сбрасываем внешний вид и ставим красную рамку
        widget.setProperty("error", "true")
        widget.style().unpolish(widget)
        widget.style().polish(widget)

```

Рисунок 21 – Функция highlight_errors()

Функция `clear_field_error()` предназначена для снятия визуального отображения ошибки с конкретного поля ввода. Внутри функции проверяется наличие ошибки, после чего поле возвращается к стандартному оформлению и принудительно обновляется его отображение. Исходный код представлен на рисунке 22.

```
#снимает красную подсветку с поля  
def clear_field_error(self, field):  
    if field.property("error") == "true":  
        # если поле свойство ошибка да, меняем на нет  
        field.setProperty("error", "false")  
        # сбрасываем стиль и обновляем  
        field.style().unpolish(field)  
        field.style().polish(field)  
        # принудительно перерисовываем поле, без задержек  
        field.update()
```

Рисунок 22 – Функция `clear_field_error()`

Класс `interface_user`, наследуемый от `Program`, реализует основную логику приложения. В нем определены функции обработки и проверки введенных данных (`start_calculation`), очистки и преобразования значений (`clean_numeric_input`), а также функции расчета кредитного графика (`build_payment_schedule`) и вывода результатов в таблицу (`update_table`). В сигнатурах ключевых функций входными параметрами являются данные, введенные пользователем: сумма кредита, срок, процентная ставка, параметры досрочного погашения и комиссий. Результатом работы функций является сформированный график платежей, а также вычисленные значения ежемесячного платежа, общей суммы выплат и переплаты.

Функция `update_comm_validator()` обновляет правила проверки значения комиссии в зависимости от выбранной единицы измерения. Если выбраны проценты, то разрешается ввод чисел от 0 до 100 с двумя знаками после запятой. Если выбрана сумма в рублях, то диапазон расширяется до

миллиарда, также с двумя знаками после запятой. Это гарантирует корректность введенных данных в поле комиссии. Исходный код представлен на рисунке 23.

```
def update_comm_validator(self):  
    # проверяем какие единицы измерения выбраны  
    if self.comm_unit_combo.currentText() == "%":  
        # если проценты то устанавливаем ограничения от 0 до 100 и два знака после запятой  
        self.comm_val_input.setValidator(QDoubleValidator(0.0, 100.0, 2))  
    else:  
        #если сумма то от 0 до миллиарда и тоже двумя знаками после запятой  
        self.comm_val_input.setValidator(QDoubleValidator(0.0, 999999999.0, 2))
```

Рисунок 23 – Функция update_comm_validator()

Функция update_term_validation() устанавливает ограничение на ввод срока кредита. Поле принимает только целые числа от 1 до 999. Исходный код представлен на рисунке 24.

```
def update_term_validation(self):  
    #для срока кредита ограничения  
    self.term_input.setValidator(QIntValidator(1, 999))
```

Рисунок 24 – Функция update_term_validation()

Функция clear_single_error(widget) убирает визуальное выделение ошибки для конкретного поля. Она сбрасывает свойство "error" и обновляет стиль виджета, возвращая его к обычному виду. Исходный код представлен на рисунке 25.

```
#сброс подсветки  
def clear_single_error(self, widget):  
    # меняем ошибку на нет, потом забываем что поле было красное и применим стандартный вид полю  
    widget.setProperty("error", "false")  
    widget.style().unpolish(widget)  
    widget.style().polish(widget)
```

Рисунок 25 - Функция clear_single_error(widget)

Функция clean_numeric_input(widget) очищает текст из поля ввода, подготавливая его к числовой обработке. Она удаляет пробелы, заменяет запятую на точку и, если есть более одной точки, склеивает их так, чтобы

оставалась только последняя, отделяющая дробную часть. Результат можно безопасно преобразовать в число для расчётов. Исходный код представлен на рисунке 26.

```
def clean_numeric_input(self, widget):  
    # берем текст из поля и убираем пробелы  
    raw_text = widget.text().strip().replace(' ', '')  
    #заменяем запятую на точку  
    clean_text = raw_text.replace(',', '.')  
    # если точек больше одной то, склеиваем все до последней точки, которая отделяет две цифры  
    if clean_text.count('.') > 1:  
        parts = clean_text.split('.')  
        return "".join(parts[:-1]) + "." + parts[-1]  
    return clean_text
```

Рисунок 26 – Функция clean_numeric_input(widget)

Функция start_calculation() выполняет основную проверку данных и расчет графика платежей. Сначала она собирает данные из всех полей, очищает и проверяет их корректность. Если есть ошибки, поля подсвечиваются красным, а пользователю показывается всплывающее окно с перечислением проблем. После успешной проверки функция рассчитывает график платежей с учётом типа ставки, способа погашения, досрочных платежей и комиссий, а затем обновляет таблицу и результаты на экране. Фрагмент кода представлен на рисунке 27.

```
def start_calculation(self):  
    # создаем пустые списки для хранения полей и текстов ошибок  
    errors_widgets = []  
    error_messages = []  
    # для объекта дата, изначально пустая  
    issue_date_obj = None  
  
    # применяем вот ту очистку поле ввода суммы  
    # убираем лишние точки запятые пробелы  
    sum_text = self.clean_numeric_input(self.sum_input)  
    # для досрочного погашения если галочка стоит  
    early_sum_text = ""  
    if self.check_early.isChecked():  
        early_sum_text = self.clean_numeric_input(self.early_sum_input)  
  
    # для комиссии если она в рублях  
    comm_sum_text = ""  
    if self.check_comm.isChecked() and self.comm_unit_combo.currentText() == "руб.":  
        comm_sum_text = self.clean_numeric_input(self.comm_val_input)
```

Рисунок 27 – Функция start_calculation()

Функция `build_payment_schedule()` формирует график платежей по кредиту. Для каждого месяца создается строка с датой, суммой платежа, частью основного долга, процентами и оставшимся долгом. Результатом является список записей, который затем отображается в таблице интерфейса. Фрагмент кода представлен на рисунке 28.

```
def build_payment_schedule(self, sum_credit, rates_history, months, payment_type, payment_day, first_only_interest,
                           early_params, comm_params):
    # список в который складываются строки таблицы
    schedule = []
    # остаток долга равен всей сумме кредита
    remaining_debt = sum_credit
    # текст из поля даты превращаем в объект от которого будем отсчитывать месяцы
    start_date = datetime.strptime(self.date_input.text(), format="%d.%m.%Y")
    # берем самую первую ставку из истории
    current_rate = rates_history[0][1]
    # расчет ежемесячной комиссии
    monthly_comm = 0
    if comm_params and not comm_params['once']:
        # если в рублях то норм, если в процентах то считаем от суммы кредита
        monthly_comm = comm_params['val'] if comm_params['unit'] == 'py6' else sum_credit * (
            comm_params['val'] / 100)
    # счетчик месяцев
    m = 1
    # пока не закончились месяцы и остаток долга больше 1 копейки
    while m <= months and remaining_debt > 0.01:
        # считаем дату платежа
        pay_date = start_date + relativedelta(months=m, day=payment_day)
```

Рисунок 28 – Функция `build_payment_schedule()`

Функция `update_table(schedule)` обновляет таблицу на экране, выводя рассчитанный график платежей. Исходный код представлен на рисунке 29.

```
# вывод в интерфейс
def update_table(self, schedule):
    # количество строк в таблице по размеру нашего списка
    self.payment_tabel.setRowCount(len(schedule))
    # строка, данные
    for row_idx, row_data in enumerate(schedule):
        # колонка, конкретное число
        for col_idx, value in enumerate(row_data):
            # форматирование чисел
            if isinstance(value, (int, float)) and col_idx > 0:
                # пробелы между тысячами
                display_text = f"{value:,.2f}".replace(_old: ",", _new: " ")
            else:
                display_text = str(value)
            # создание ячейки таблицы
            item = QTableWidgetItem(display_text)
            # выравниваем текст по центру
            item.setTextAlignment(Qt.AlignmentFlag.AlignCenter)
            # вставляем ячейку на определенное место
            self.payment_tabel.setItem(row_idx, col_idx, item)
```

Рисунок 29 – Функция `update_table(schedule)`

Функция `handle_early_check(state)` реагирует на изменение состояния чекбокса досрочного погашения. Фрагмент кода представлен на рисунке 30.

```
# чекбокс досрочки
def handle_early_check(self, state):
    # если галочка снята
    if state == 0:
        # очищаем поле и убираем красную подсветку
        self.early_sum_input.clear()
        self.clear_single_error(self.early_sum_input)
```

Рисунок 30 – Функция `handle_early_check(state)`

Функция `handle_comm_check(state)` реагирует на изменение состояния чекбокса комиссии. Исходный код представлен на рисунке 31.

```
# тоже самое с комиссиями
def handle_comm_check(self, state):
    if state == 0:
        self.comm_val_input.clear()
        self.clear_single_error(self.comm_val_input)
```

Рисунок 31 – Функция `handle_comm_check(state)`

Функция `clear_all_data()` сбрасывает все результаты расчетов и очищает экран. Исходный код представлен на рисунке 32.

```
# метод очистки всех результатов расчета на экране
def clear_all_data(self):
    super().clear_all_data()
    # обнуляем метки с результатами
    self.res_payment.setText("0.00 py6.")
    self.res_total.setText("0.00 py6.")
    self.res_overpay.setText("0.00 py6.")
```

Рисунок 32 – Функция `clear_all_data()`

4.3.2 Разработка пользовательского интерфейса

Графический интерфейс приложения реализован на базе библиотеки PyQt6, что позволило создать современный, интуитивно понятный и функциональный дизайн с использованием продвинутой системы стилей. Окно программы, представленный на рисунке 33 имеет продуманную вертикальную структуру и выполнено в спокойной светлой цветовой гамме с акцентами зеленого цвета.

Основные компоненты интерфейса:

- Заголовок и контейнеры: Верхняя часть окна и логические блоки выделены с помощью виджетов QFrame. Это визуально отделяет области ввода данных от итоговых результатов, создавая четкую иерархию элементов.
- Блок ввода данных: состоит из полей ввода QLineEdit. В некоторых полях реализован механизм Placeholder (подсказка): серый текст с примерами (например, «0.00 через точку»), который автоматически исчезает при начале ввода, подготавливая поле для работы.
- Валидация и контроль: для защиты от некорректного ввода (букв, знаков или недопустимых значений) применены объекты QIntValidator и QDoubleValidator. В частности, для поля процентной ставки программно установлено ограничение в диапазоне от 1% до 100%.
- Окно вывода результатов: Реализовано на базе виджета QTableWidgetItem. Использование табличной структуры обеспечивает строгое выравнивание данных по столбцам (дата, платеж, проценты, остаток долга). Система форматирования чисел разделяет тысячи пробелами для удобства чтения.
- Интерактивные переключатели: Блоки выбора построены на кнопках QPushButton со свойством Checkable. Это обеспечивает визуальный отклик: активная настройка остается «зажатой» и подсвечивается цветом.

– Система обратной связи: Реализована динамическая подсветка ошибок. При пропуске поля или некорректном значении виджет окрашивается в красный цвет. При начале исправления данных поле мгновенно возвращается к стандартному стилю с помощью методов `unpolish` и `polish`.

Управляющие кнопки:

– «Рассчитать» – инициирует алгоритм обработки введенных данных, формирует график платежей и выводит итоговые суммы (общая выплата, переплата).

– «Очистить» – мгновенно сбрасывает все поля ввода, очищает таблицу результатов и снимает индикацию ошибок.

Главное окно реализовано, которое через класс `QMainWindow`. Весь функционал управления связан с логикой программы через механизм сигналов, что делает отклик приложения мгновенным.

The screenshot shows a Qt-based application window with a light green theme. It is divided into several sections for inputting credit parameters and displaying results.

Параметры кредита (Credit Parameters):

- Сумма кредита:
- Дата выдачи:
- Срок кредита: мес.
- Ставка кредита: %
- Вид платежа:
- День платежей:
- Первый платеж:

Досрочное погашение (Early Repayment):

- Сумма досрочного платежа:
- Тип погашения:
- Частота:
- Дата:

Комиссии (Commissions):

- Тип комиссии:
- Введите значение: руб.

Результат (Result):

- Ежемесячный платеж:
- Общая сумма платежа:
- Переплата:

График платежей (Payment Schedule):

Дата	Платеж	Остаток	Проценты	Остаток

At the bottom, there are two large buttons: **Рассчитать** (Calculate) in green and **Очистить** (Clear) in red.

Рисунок 33 – Основное окно приложения

Пример расчета представлен на рисунке 34.

Параметры кредита

Сумма кредита: 100000

Дата выдачи: 04.04.2026

Срок кредита: 12 мес.

Ставка кредита: 12

Вид платежа: **Ануитетный**

День платежей: В день выдачи кредита

Первый платеж: Только проценты

Досрочное погашение

Сумма досрочного платежа: 0.00

Тип погашения: Снизить платеж, Снизить срок

Частота: Единоновременно, Раз в месяц, Раз в год

Дата: ..

Комиссии

Единовременная, Ежемесячная

Введите значение: руб.

Рассчитать **Очистить**

Результат:

Ежемесячный платеж: 1000.00 руб.

Общая сумма платежа: 107099.51 руб.

Переплата: 7099.51 руб.

График платежей

Дата	Платеж	Остаток	Проценты	Остаток
04.05.2026	1 000.00	0.00	1 000.00	100 000.00
04.06.2026	9 645.41	8 645.41	1 000.00	91 354.59
04.07.2026	9 645.41	8 731.86	913.55	82 622.73
04.08.2026	9 645.41	8 819.18	826.23	73 803.55
04.09.2026	9 645.41	8 907.37	738.04	64 896.18
04.10.2026	9 645.41	8 996.45	648.96	55 899.73
04.11.2026	9 645.41	9 086.41	559.00	46 813.32
04.12.2026	9 645.41	9 177.27	468.13	37 636.05
04.01.2027	9 645.41	9 269.05	376.36	28 367.00
04.02.2027	9 645.41	9 361.74	283.67	19 005.26
04.03.2027	9 645.41	9 455.35	190.05	9 549.91
04.04.2027	9 645.41	9 549.91	95.50	0.00

Рисунок 34 – Пример расчета

Вывод: в рамках раздела реализовано программное обеспечение «Кредитный калькулятор» на языке Python. Разработанная архитектура базируется на объектно-ориентированном подходе и использовании современной графической библиотеки PyQt6.

Созданный пользовательский интерфейс обеспечивает удобство работы благодаря внедрению системы валидации данных, интерактивных подсказок и динамической индикации ошибок. Использование виджета QTableWidget позволило достичь наглядной визуализации детального графика платежей.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Welcome to Python.org., URL: <https://www.python.org/> (дата обращения: 24.01.2026).
2. Download PyCharm: The Python IDE for data science and web development by JetBrains., URL: <https://www.jetbrains.com/pycharm/download/> (дата обращения: 24.01.2026).
3. Что такое кредит: понятие, виды и классификация : электрон. дан., URL: https://rencredit.ru/blog/credits/chto-takoe-kredit-ponyatie-vidy-i-klassifikatsiya/?utm_referrer=https%3a%2f%2fyandex.ru%2f (дата обращения: 24.01.2026).
4. Что такое кредит, виды, функции и принцип работы банковских кредитов : электрон. дан., URL: <https://www.tbank.ru/bank/help/loans/educate/main-points/credit/> (дата обращения: 24.01.2026).
5. Что такое кредит простыми словами : электрон. дан., URL: <https://www.sberbank.ru/ru/person/blog/chto-takoe-kredit-prostimi-slovami> (дата обращения: 24.01.2026).
6. Удовлетворяя потребность в вычислениях : электрон. дан., URL: <https://habr.com/ru/articles/859708/> (дата обращения: 24.01.2026).
7. Калькулятор : электрон. дан., URL: <https://ru.ruwiki.ru/wiki/Калькулятор> (дата обращения: 24.01.2026).

ПРИЛОЖЕНИЕ А

Модуль интерфейса

```
from PyQt6.QtWidgets import QApplication, QMainWindow, QWidget, QVBoxLayout, QPushButton,
QFrame, QLabel, QLineEdit, \
    QHBoxLayout, QGridLayout, ButtonGroup, QTableWidgetItem, QHeaderView,
QScrollArea, QComboBox, \
    QCheckBox, QSpinBox, QMessageBox, QToolTip
from PyQt6.QtCore import Qt, QLocale, QRegularExpression, QPoint
from PyQt6.QtGui import QDoubleValidator, QIntValidator, QRegularExpressionValidator, QColor
import sys

class Program(QMainWindow):
    def __init__(self):
        super().__init__()
        #валидация для дробных чисел
        self.numeric_validator = QDoubleValidator(0.0, 1000000000.0, 2)
        # вид записи чтоб был стандартным без е и тл
        self.numeric_validator.setNotation(QDoubleValidator.Notation.StandardNotation)
        #валидация ввода точки, а не запятой
        self.numeric_validator.setLocale(QLocale(QLocale.Language.English,
        QLocale.Country.UnitedStates))

        # создаем кнопки и применяем к ним дизайн
        self.init_ui()
        self.apply_styles()
        #все поля ввода
        self.all_inputs = [
            self.sum_input, self.date_input, self.term_input,
            self.rate_input, self.rate_input_fixed,
            self.early_sum_input, self.early_date_input, self.comm_val_input,
            self.custom_day_input
        ]
        # кроме полей даты и срока, устанавливаем валидацию на ввод только чисел
        for field in self.all_inputs:
            if field not in [self.date_input, self.term_input, self.early_date_input]:
                field.setValidator(self.numeric_validator)

            #Снимаем красную штуку, как только пользователь начал вводить чтото
            field.textChanged.connect(lambda _, f=field: self.clear_field_error(f))
        # для срока кредита разрешен вво только целых числе от 1 до 600
        self.term_input.setValidator(QIntValidator(1, 600))

        #привязываем функции к кнопкам
        self.btn_clear.clicked.connect(self.clear_all_data)
        self.check_early.stateChanged.connect(self.toggle_sections)
        self.check_comm.stateChanged.connect(self.toggle_sections)
        self.btn_fixed.clicked.connect(self.on_rate_type_changed)
        self.btn_variable.clicked.connect(self.on_rate_type_changed)
        self.btn_add_rate.clicked.connect(self.add_rate_row)

        #блокируем поля досрочки и комиссий
        self.toggle_sections()
        #делаем сразу фиксированную ставку для удобства ну и так как она чаще всего выбирается
        self.btn_fixed.setChecked(True)
        # это для видимости полей при выборе ставки
        self.on_rate_type_changed()
        # белая таблица сразу
        self.clear_table_to_default()

        # функция которая запрещает начинать ввод с точки или запятой
        def validate_input_start(self, widget):
            text = widget.text()
            if text.startswith('.') or text.startswith(','):
                # убирает точку и запятую
                widget.setText(text.lstrip('.,'))
                #сообщение об ошибке
                QToolTip.showText(
                    widget.mapToGlobal(QPoint(0, widget.height())) ,
                    "Число не может начинаться с разделителя",
                    widget
                )
```

```

# создание кнопки со знаком вопроса
def create_help_button(self, text):
    btn = QPushButton("?")
    btn.setFixedSize(22, 22)
    btn.setCursor(Qt.CursorShape.PointingHandCursor)
    btn.setObjectName("helpButton")
    # убираем всплывающую подсказку (она появляется если долго держать курсор на кнопке)
    btn.setToolTip("")
    # а вот тут создаем подсказку по клику
    btn.clicked.connect(lambda: QToolTip.showText(
        btn.mapToGlobal(QPoint(25, 0)),
        text,
        btn
    ))

    return btn

# делает сразу 10 пустых строк, для обозначения что это таблица
def clear_table_to_default(self):
    self.payment_tabel.setRowCount(10)
    # проходимся по строкам и столбцам
    for i in range(10):
        for j in range(self.payment_tabel.columnCount()):
            # создаем объект, те ячейку и записываем в нее пустую строку
            item = QTableWidgetItem("")
            item.setBackground(QColor("white"))
            # созданную ячейку кладем в таблицу на пересечение строки и столбца
            self.payment_tabel.setItem(i, j, item)

# очищаем ошибки
def clear_all_errors(self):
    # создаем список с основными полями
    all_to_check = [self.sum_input, self.term_input, self.rate_input,
                    self.rate_input_fixed, self.date_input, self.custom_day_input]
    # передается пустой список для того чтобы убрать красную обводку
    self.highlight_errors([])

# кнопка очистить
def clear_all_data(self):
    # проходим по списку всех текстовых полей
    for field in self.all_inputs:
        field.clear()
    # снимаем красные поля
    self.clear_all_errors()
    # и делаем таблицу по дефолту
    self.clear_table_to_default()

# очистка полей в части результат
self.res_payment.clear()
self.res_total.clear()
self.res_overpay.clear()

# сброс чекбоксов
self.check_early.setChecked(False)
self.check_comm.setChecked(False)

# сброс для групп кнопок
for group in [self.type_group, self.day_group, self.furst_pay_group, self.early_group,
self.freq_group,
                self.comm_group]:
    # временно отключаем эксклюзивность, те не иметь нажатых кнопок
    group.setExclusive(False)
    # для каждой кнопки из групп
    for btn in group.buttons():
        # выключаем нажатие кнопки
        btn.setChecked(False)
    # подключаем эксклюзивность в группу
    group.setExclusive(True)

# сразу ставим рабочую кнопку ставки фиксированная
self.btn_fixed.setChecked(True)
# управление полями в зависимости от ставки
self.on_rate_type_changed()

# чекбоксы
def toggle_sections(self):
    # досрочка, проверка на то, стоит ли галочка

```

```

is_early = self.check_early.isChecked()
# создаем список всех полей которые находяся в досрочке
early_widgets = [
    self.early_sum_input, self.btn_reduce_sum, self.btn_reduce_term,
    self.btn_once, self.btn_monthly, self.btn_yearly, self.early_date_input
]
# проходим по списку элементов
for w in early_widgets:
    # включаем или выключаем элемент
    w.setEnabled(is_early)
    # если галочки нет, то убираем красную рамку
    if not is_early:
        self.clear_field_error(w)

# чекбокс у комиссии
is_comm = self.check_comm.isChecked()
comm_widgets = [
    self.btn_comm_once, self.btn_comm_month,
    self.comm_val_input, self.comm_unit_combo
]
for w in comm_widgets:
    w.setEnabled(is_comm)
    if not is_comm:
        self.clear_field_error(w)

def init_ui(self):
    self.setWindowTitle("Калькулятор")
    #добавляем скролл
    self.main_scroll = QScrollArea()
    # содержимое подстраивается под размер
    self.main_scroll.setWidgetResizable(True)
    # минус лишняя рамка вокруг скролла
    self.main_scroll.setFrameShape(QFrame.Shape.NoFrame)
    # делаем его главным элементом окна
    self.setCentralWidget(self.main_scroll)
    # создаем виджет, как подложку под все остальное и добавляем ее внутрь скрола
    self.scroll_content = QWidget()
    self.main_scroll.setWidget(self.scroll_content)
    # создаем горизонтальный макет и делаем там отступы
    self.main_layout = QHBoxLayout(self.scroll_content)
    self.main_layout.setContentsMargins(20, 20, 20, 20)
    # отступы между левой и правой панелью
    self.main_layout.setSpacing(20)

    # левая колонка и внутри отступы между параметрами досрочкой и комиссией
    self.left_panel = QVBoxLayout()
    self.left_panel.setSpacing(15)

    # создаем главное окно параметра
    self.main_frame = QFrame()
    self.main_frame.setObjectName("MainFrame")
    self.main_frame.setFixedWidth(500)

    layout = QVBoxLayout(self.main_frame)
    layout.setSpacing(15)
    layout.setContentsMargins(20, 20, 20, 20)

    header = QLabel("Параметры кредита")
    header.setObjectName("Header")
    layout.addWidget(header)

    layout.addWidget(QLabel('Сумма кредита'))
    self.sum_input = QLineEdit()
    self.sum_input.setPlaceholderText("0.00 (через точку!)")
    layout.addWidget(self.sum_input)

    layout.addWidget(QLabel("Дата выдачи"))
    self.date_input = QLineEdit()
    self.date_input.setPlaceholderText("ДД.ММ.ГГГГ")
    self.date_input.setInputMask("99.99.9999")
    layout.addWidget(self.date_input)

    layout.addWidget(QLabel("Срок кредита"))
    term_layout = QHBoxLayout()
    self.term_input = QLineEdit()

    self.term_unit_combo = QComboBox()

```

```

self.term_unit_combo.addItem("мес.", "лет"])
self.term_unit_combo.setFixedWidth(80)

term_layout.addWidget(self.term_input)
term_layout.addWidget(self.term_unit_combo)
layout.addLayout(term_layout)

layout.addWidget(QLabel("Ставка кредита"))
rate_buttons = QHBoxLayout()
self.btn_fixed = QPushButton("Фиксированная")
self.btn_variable = QPushButton("Изменяемая")
self.btn_fixed.setCheckable(True)
self.btn_variable.setCheckable(True)
self.rate_group = ButtonGroup(self)
self.rate_group.addButton(self.btn_fixed)
self.rate_group.addButton(self.btn_variable)
rate_buttons.addWidget(self.btn_fixed)
rate_buttons.addWidget(self.btn_variable)
layout.addLayout(rate_buttons)

#СТАВКИ
self.rate_container = QFrame()
self.rate_container.setObjectName("RateContainer")
self.rate_container.hide()

self.rate_changes_layout = QVBoxLayout(self.rate_container)
self.rate_changes_layout.setSpacing(0)
self.rate_changes_layout.setContentsMargins(0, 0, 0, 0)

first_row = QHBoxLayout()
first_row.setContentsMargins(15, 10, 15, 10)
lbl_start = QLabel("Дата выдачи")
lbl_start.setFixedWidth(120)
self.rate_input = QLineEdit()
self.rate_input.setPlaceholderText("0.00%")
self.rate_input.setAlignment(Qt.AlignmentFlag.AlignCenter)

first_row.addWidget(lbl_start)
first_row.addWidget(self.rate_input)
first_row.addSpacing(40)
self.rate_changes_layout.addLayout(first_row)

layout.addWidget(self.rate_container)

self.btn_add_rate = QPushButton("ДОБАВИТЬ")
self.btn_add_rate.setObjectName("addRateBtn")
self.btn_add_rate.setCursor(Qt.CursorShape.PointingHandCursor)
self.btn_add_rate.hide()
layout.addWidget(self.btn_add_rate)

#поле для фикс ставки
self.rate_input_fixed = QLineEdit()
self.rate_input_fixed.setPlaceholderText("%")
layout.addWidget(self.rate_input_fixed)

type_label_layout = QHBoxLayout()
type_label_layout.addWidget(QLabel("Вид платежа"))
self.help_type = self.create_help_button(
    "Аннуитетный — платежи равны весь срок.\nДифференцированный — платеж уменьшается к
концу.")
type_label_layout.addWidget(self.help_type)
type_label_layout.addStretch()
layout.addLayout(type_label_layout)
type_buttons = QHBoxLayout()
self.btn_annuity = QPushButton("Аннуитетный")
self.btn_diff = QPushButton("Дифференцированный")
self.btn_annuity.setCheckable(True)
self.btn_diff.setCheckable(True)
self.type_group = ButtonGroup(self)
self.type_group.addButton(self.btn_annuity)
self.type_group.addButton(self.btn_diff)

type_buttons.addWidget(self.btn_annuity)
type_buttons.addWidget(self.btn_diff)
layout.addLayout(type_buttons)

day_label_layout = QHBoxLayout()

```

```

day_label_layout.addWidget(QLabel("День платежей"))
self.help_day = self.create_help_button(
    "Выберите дату ежемесячного списания.\nЕсли день 31-й, а в месяце 30 дней –
    спишется в последний день.")
day_label_layout.addWidget(self.help_day)
day_label_layout.addStretch()
layout.addLayout(day_label_layout)
# создаем сетку
day_grid = QGridLayout()

self.btn_issue_day = QPushButton("В день выдачи\nкредита")
self.btn_issue_day.setObjectName("smallBtn")
self.btn_issue_day.setCheckable(True)

self.btn_last_day = QPushButton("В последний день\nмесяца")
self.btn_last_day.setObjectName("smallBtn")
self.btn_last_day.setCheckable(True)

self.custom_day_input = QLineEdit()
self.custom_day_input.setPlaceholderText("Свой день")
self.custom_day_input.setAlignment(Qt.AlignmentFlag.AlignCenter)
self.custom_day_input.setFixedWidth(100)
self.custom_day_input.setMaxLength(2)
# ввод чисел от 1 до 31 РЕШИТЬ ЧТОТО С ЭТИМ
reg_exp = QRegularExpression("^(?:[1-9]|[12][0-9]|3[01])$")
validator = QRegularExpressionValidator(reg_exp, self.custom_day_input)
self.custom_day_input.setValidator(validator)
# создаем группу кнопок и делаем так чтобы была нажата только одна
self.day_group = QButtonGroup(self)
self.day_group.addButton(self.btn_issue_day)
self.day_group.addButton(self.btn_last_day)
self.day_group.setExclusive(True)

#если начали писать сввой день
self.custom_day_input.textChanged.connect(
    lambda: [
        self.day_group.setExclusive(False), # разрешаем группе иметь ниодной нажатой
        [b.setChecked(False) for b in self.day_group.buttons()], #если были выбраны
        self.day_group.setExclusive(True) #включаем режим выбора одной кнопки
    ] if self.custom_day_input.text() else None) #делаем только в том случае если поле
не пустое
# если кликнули на любую кнопку в группе, то очищаем поле свой день
self.day_group.buttonClicked.connect(lambda: self.custom_day_input.clear())
# размещаем кнопки и поле ввода в сетке
day_grid.addWidget(self.btn_issue_day, 0, 0)
day_grid.addWidget(self.btn_last_day, 0, 1)
day_grid.addWidget(self.custom_day_input, 0, 2)
# добавляем сетку в макет
layout.addLayout(day_grid)

layout.addWidget(QLabel("Первый платеж"))
first_pay_buttons = QHBoxLayout()
self.btn_interest_only = QPushButton("Только проценты")
self.btn_full_pay = QPushButton("Проценты и часть\nосновного долга")
self.btn_full_pay.setObjectName("smallBtn")
first_pay_buttons.addWidget(self.btn_interest_only)
first_pay_buttons.addWidget(self.btn_full_pay)
self.btn_interest_only.setCheckable(True)
self.btn_full_pay.setCheckable(True)
self.first_pay_group = QButtonGroup(self)
self.first_pay_group.addButton(self.btn_interest_only)
self.first_pay_group.addButton(self.btn_full_pay)
layout.addLayout(first_pay_buttons)

#Досрочное погашение
self.early_pay_frame = QFrame()
self.early_pay_frame.setObjectName("MainFrame")
self.early_pay_frame.setFixedWidth(500)
early_layout = QVBoxLayout(self.early_pay_frame)
early_layout.setContentsMargins(20, 20, 20, 20)
early_layout.setSpacing(10)

early_header = QLabel("Досрочное погашение")
early_header.setObjectName("Header")

```

```

#чекбокс досрочки
self.check_early = QCheckBox()
self.check_early.setFixedSize(25, 25)

early_header_layout = QHBoxLayout()
early_header_layout.addWidget(early_header)
self.help_early = self.create_help_button(
    "Снижение платежа – сумма в месяц станет меньше.\nСнижение срока – быстрее
закроете кредит.")
early_header_layout.addWidget(self.help_early)
early_header_layout.addWidget(self.check_early)
early_header_layout.addStretch()
early_layout.addLayout(early_header_layout)

early_layout.addWidget(QLabel("Сумма досрочного платежа"))
self.early_sum_input = QLineEdit()
self.early_sum_input.setPlaceholderText("0.00")
early_layout.addWidget(self.early_sum_input)

early_layout.addWidget(QLabel("Тип погашения"))
early_type_btns = QHBoxLayout()
self.btn_reduce_sum = QPushButton("Снизить платёж")
self.btn_reduce_term = QPushButton("Снизить срок")
self.btn_reduce_sum.setCheckable(True)
self.btn_reduce_term.setCheckable(True)
self.early_group = QButtonGroup(self)
self.early_group.addButton(self.btn_reduce_sum)
self.early_group.addButton(self.btn_reduce_term)
early_type_btns.addWidget(self.btn_reduce_sum)
early_type_btns.addWidget(self.btn_reduce_term)
early_layout.addLayout(early_type_btns)

early_layout.addWidget(QLabel("Частота"))
freq_btns = QHBoxLayout()
self.btn_once = QPushButton("Единовременно")
self.btn_monthly = QPushButton("Раз в месяц")
self.btn_yearly = QPushButton("Раз в год")
self.freq_group = QButtonGroup(self)
for btn in [self.btn_once, self.btn_monthly, self.btn_yearly]:
    # делаем кнопки кликабельными
    btn.setCheckable(True)
    # добавляем в группу
    self.freq_group.addButton(btn)
    # и добавляем в горизонтальный ряд а потом в общий макет
    freq_btns.addWidget(btn)
early_layout.addLayout(freq_btns)

early_layout.addWidget(QLabel("Дата"))
self.early_date_input = QLineEdit()
self.early_date_input.setPlaceholderText("ДД.ММ.ГГГГ")
self.early_date_input.setInputMask("99.99.9999")
early_layout.addWidget(self.early_date_input)

#Комиссии
self.comm_frame = QFrame()
self.comm_frame.setObjectName("MainFrame")
self.comm_frame.setFixedWidth(500)
comm_layout = QVBoxLayout(self.comm_frame)
comm_layout.setContentsMargins(20, 20, 20, 20)
comm_layout.setSpacing(10)

comm_header = QLabel("Комиссии")
comm_header.setObjectName("Header")

#чекбокс комиссий
self.check_comm = QCheckBox()
self.check_comm.setFixedSize(25, 25)

comm_header_layout = QHBoxLayout()
comm_header_layout.addWidget(comm_header)
self.help_comm = self.create_help_button(
    "Единовременная – платится один раз при выдаче.\nЕжемесячная – добавляется к
каждому платежу.")
comm_header_layout.addWidget(self.help_comm)
comm_header_layout.addWidget(self.check_comm)
comm_header_layout.addStretch()
comm_layout.addLayout(comm_header_layout)

```

```

comm_type_btns = QHBoxLayout()
self.btn_comm_once = QPushButton("Единовременная")
self.btn_comm_month = QPushButton("Ежемесячная")
self.btn_comm_once.setCheckable(True)
self.btn_comm_month.setCheckable(True)
self.comm_group = ButtonGroup(self)
self.comm_group.addButton(self.btn_comm_once)
self.comm_group.addButton(self.btn_comm_month)
comm_type_btns.addWidget(self.btn_comm_once)
comm_type_btns.addWidget(self.btn_comm_month)
comm_layout.addLayout(comm_type_btns)

comm_inputs = QHBoxLayout()
self.comm_val_input = QLineEdit()
self.comm_val_input.setPlaceholderText("Введите значение")
self.comm_unit_combo = QComboBox()
self.comm_unit_combo.addItem("руб", "%")
self.comm_unit_combo.setFixedWidth(80)
comm_inputs.addWidget(self.comm_val_input)
comm_inputs.addWidget(self.comm_unit_combo)
comm_layout.addLayout(comm_inputs)

#собираем в левую панель
self.left_panel.addWidget(self.main_frame)
self.left_panel.addWidget(self.early_pay_frame)
self.left_panel.addWidget(self.comm_frame)

#Кнопки рассчитать и очистить
result_buttons = QHBoxLayout()
self.btn_calculate = QPushButton("Рассчитать")
self.btn_calculate.setObjectName("calculateBTN")
self.btn_clear = QPushButton("Очистить")
self.btn_clear.setObjectName("clearBTN")
result_buttons.addWidget(self.btn_calculate)
result_buttons.addWidget(self.btn_clear)
self.left_panel.addLayout(result_buttons)
self.left_panel.addStretch()

#Фрейм для результатов
self.result_frame = QFrame()
self.result_frame.setObjectName("resultFrame")
self.result_frame.setFixedHeight(200)
result_layout = QVBoxLayout(self.result_frame)
result_layout.setContentsMargins(20, 20, 20, 20)

self.result_title = QLabel("Результат:")
self.result_title.setObjectName("Result")
result_layout.addWidget(self.result_title)

def create_result_row(text):
    row = QHBoxLayout()
    label = QLabel(text)
    display = QLineEdit()
    display.setReadOnly(True)
    display.setFixedWidth(700)
    row.addWidget(label)
    row.addWidget(display)

    result_layout.addLayout(row)
    return display

self.res_payment = create_result_row("Ежемесячный платеж:")
self.res_total = create_result_row("Общая сумма платежа:")
self.res_overpay = create_result_row("Переплата:")

#Фрейм с графиком платежей
self.table_frame = QFrame()
self.table_frame.setObjectName("tabelname")
table_layout = QVBoxLayout(self.table_frame)

table_header = QLabel("График платежей")
table_header.setObjectName("tabelheader")
# выравниваем по середине
table_header.setAlignment(Qt.AlignmentFlag.AlignCenter)
table_layout.addWidget(table_header)

```



```

self.payment_table = QTableWidgetItem()
# скрываем номера строк
self.payment_table.verticalHeader().setVisible(False)
# запрещаем печатать текст в ячейках
self.payment_table.setEditTriggers(QTableWidgetItem.EditTrigger.NoEditTriggers)
# создаем 5 колоно и пишем у них названия
self.payment_table.setColumnCount(5)
self.payment_table.setHorizontalHeaderLabels(["Дата", "Платеж", "Осн.долг",
"Проценты", "Остаток"])

#строки в таблице
# берем названия и растягиваем колонки во всю ширину таблицы
header = self.payment_table.horizontalHeader()
header.setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
# добавляем таблицу в макет и создаем сразу 10 пустых строк
table_layout.addWidget(self.payment_table)
self.payment_table.setRowCount(10)

#правая колонка
self.right_panel = QVBoxLayout()
self.right_panel.addWidget(self.result_frame)
self.right_panel.addWidget(self.table_frame)

#Сборка левой и правой пнели
self.main_layout.addLayout(self.left_panel)
self.main_layout.addLayout(self.right_panel)

def apply_styles(self):
self.setStyleSheet("""
    #MainFrame, #resultFrame, #tabelname {
        background-color: #E2F9E8;
        border-radius: 20px;
        border: 2.5px solid #74c43f;
    }

    #Header, #tabelheader, #Result {
        font-size: 18px;
        font-weight: bold;
        color: #333;
    }

    #calculateBTN {
        background: #90f59a;
        min-height: 40px;
        font-weight: bold;
        border-radius: 10px;
    }

    #clearBTN {
        background: #f66565;
        color: white;
        min-height: 40px;
        font-weight: bold;
        border-radius: 10px;
    }

    #helpButton {
        border-radius: 11px;
        background-color: #D1D1E9;
        color: #4A4A4A;
        font-weight: bold;
        border: none;
        padding: 0px;
    }

    #helpButton:hover {
        background-color: #B1B1D9;
    }

    QToolTip {
        background-color: #E2F9E8;
        color: #333333;
        border: 1.5px solid #74c43f;
        padding: 8px;
        font-size: 13px;
    }

```

```

QLabel {
    font-size: 14px;
    color: black;
    margin-top: 5px;
    background: transparent;
}

QLineEdit, QPushButton, QComboBox, QSpinBox {
    border: none;
    border-radius: 10px;
    background-color: white;
    padding: 5px;
    font-size: 14px;
    color: #333;
    min-height: 25px;
}

QLineEdit[error="true"] {
    border: 2px solid #f66565;
    background-color: #fff0f0;
}

QPushButton:checked {
    background-color: #B4F0C9;
    color: black;
    font-weight: bold;
}

QPushButton#smallBtn {
    font-size: 10px;
}

QCheckBox::indicator {
    width: 20px;
    height: 20px;
    border: 2px solid #74c43f;
    border-radius: 2px;
    background-color: white;
}

QCheckBox::indicator:checked {
    background-color: #90f59a;
    border: 2px solid #2e7d32;
}

QTableWidget{
    background-color: transparent;
    border: none;
    gridline-color: transparent;
    alternate-background-color: #F0F0F0;
    outline: none;
}

QTableWidget::item {
    background-color: white;
    color: black;
    padding: 10px;
    border: none;
}

QHeaderView::section {
    background-color: #B4F0C9;
    border: none;
    font-weight: bold;
    font-size: 13px;
    padding: 5px;
    color: black;
}

QLineEdit:disabled, QPushButton:disabled, QComboBox:disabled {
    background-color: #f0f0f0;
    color: #a0a0a0;
}

QMessageBox {
    background-color: #E2F9E8;
    border: 2px solid #74c43f;
}

QMessageBox QPushButton {
    background-color: #90f59a;
    border-radius: 8px;
}

```

```

        padding: 5px 15px;
        font-weight: bold;
    }

    """)

# для изменения ставки
def add_rate_row(self):
    row_widget = QWidget()
    row_layout = QHBoxLayout(row_widget)
    row_layout.setContentsMargins(15, 10, 15, 10)

    date_edit = QLineEdit()
    date_edit.setPlaceholderText("ДД.ММ.ГГГГ")
    date_edit.setInputMask("99.99.9999")
    # убираем красную рамку если пользователь начал что-то писать
    date_edit.textChanged.connect(lambda: self.clear_field_error(date_edit))

    rate_edit = QLineEdit()
    rate_edit.setPlaceholderText("0.00%")
    # текст по центру поля, разрешаем писать только числа
    rate_edit.setAlignment(Qt.AlignmentFlag.AlignCenter)
    rate_edit.setValidator(self.numeric_validator)
    # очищаем поля при изменении текста и ставим проверку на ввод, чтобы не начинался с .
    или ,
    rate_edit.textChanged.connect(lambda: self.clear_field_error(rate_edit))
    rate_edit.textChanged.connect(lambda: self.validate_input_start(rate_edit))

    del_btn = QPushButton("X")
    del_btn.setFixedSize(30, 30)
    # если нажимаем удаляем весь row_widget
    del_btn.clicked.connect(lambda: row_widget.deleteLater())
    # добавляем все элементы и добавляем в вертикальный макет
    row_layout.addWidget(date_edit)
    row_layout.addWidget(rate_edit)
    row_layout.addWidget(del_btn)
    self.rate_changes_layout.addWidget(row_widget)

#кнопка добавить только при изменяемой ставке
def on_rate_type_changed(self):
    # нажата ли сейчас кнопка изменяемая
    is_var = self.btn_variable.isChecked()
    # если да, то показываем контейнер с датами и ставкой и кнопку добавить
    self.rate_container.setVisible(is_var)
    self.btn_add_rate.setVisible(is_var)
    # поле для фиксированной показываем если не нажата изменяемая ставка
    self.rate_input_fixed.setVisible(not is_var)
    # если вернулся к фиксированной ставке
    if not is_var:
        #Очищает строки и возвращает фиксированную ставку
        # идем с конца списка, удаляем все кроме первой строк
        for i in reversed(range(1, self.rate_changes_layout.count())):
            # берем строку под номером i и если она нашлась то удаляем
            widget = self.rate_changes_layout.itemAt(i).widget()
            if widget: widget.deleteLater()

def highlight_errors(self, widgets):
    #проходим по всем полям ввола
    for field in self.all_inputs:
        # сначала убираем ошибку, после заставляем забыть как выглядим поле и заново
        делаем проверку
        field.setProperty("error", "false")
        field.style().unpolish(field)
        field.style().polish(field)

    #проходим по полям в которых есть ошибка
    for widget in widgets:
        # ошибка-да, сбрасываем внешний вид и ставим красную рамку
        widget.setProperty("error", "true")
        widget.style().unpolish(widget)
        widget.style().polish(widget)

#снимает красную подсветку с поля
def clear_field_error(self, field):
    if field.property("error") == "true":
        # если поле свойство ошибка да, меняем на нет

```

```

field.setProperty("error", "false")
# сбрасываем стиль и обновляем
field.style().unpolish(field)
field.style().polish(field)
# принудительно перерисовываем поле, без задержек
field.update()

```

Модуль логики

```

import sys
from PyQt6.QtWidgets import QApplication, QTableWidgetItem, QMessageBox, QLineEdit
from interface import Program
from PyQt6.QtCore import Qt
from PyQt6.QtGui import QIntValidator, QDoubleValidator
from datetime import datetime
from dateutil.relativedelta import relativedelta

class interface user(Program):
    def __init__(self):
        super().__init__()
        # выпадающий список для мес/лет
        self.term_unit_combo.currentIndexChanged.connect(self.update_term_validation)
        # выпадающий список для комиссий руб/проценты
        self.comm_unit_combo.currentTextChanged.connect(self.update_comm_validator)

        # список всех полей ввода для проверки, чтобы в начале не ставилась точка
        money_fields = [
            self.sum_input,
            self.rate_input_fixed,
            self.rate_input,
            self.early_sum_input,
            self.comm_val_input
        ]
        # для каждого поля в списке делаем проверку
        for field in money_fields:
            # validate_input_start удалит точку или запятую если ввели первой
            field.textChanged.connect(lambda _, f=field: self.validate_input_start(f))
        # вызываем функции сразу, чтобы лимиты работали сразу
        self.update_term_validation()
        self.update_comm_validator()
        # кнопку рассчитать связываем с функцией расчета
        self.btn_calculate.clicked.connect(self.start_calculation)
        # при запуске теперь всегда будет сегодняшняя дата
        self.date_input.setText(datetime.now().strftime("%d.%m.%Y"))

        # если начинается ввод в поле то убираем красную подсветку
        self.sum_input.textChanged.connect(lambda: self.clear_single_error(self.sum_input))
        self.term_input.textChanged.connect(lambda: self.clear_single_error(self.term_input))
        self.custom_day_input.textChanged.connect(lambda:
self.clear_single_error(self.custom_day_input))
        # если нажали на кнопки выбора дня тоже убираем ошибку
        self.btn_issue_day.clicked.connect(lambda:
self.clear_single_error(self.custom_day_input))
        self.btn_last_day.clicked.connect(lambda:
self.clear_single_error(self.custom_day_input))
        # управление чекбоксами, вкл или выкл ввод параметров
        self.check_early.stateChanged.connect(self.handle_early_check)
        self.check_comm.stateChanged.connect(self.handle_comm_check)

    def update_comm_validator(self):
        # проверяем какие единицы измерения выбраны
        if self.comm_unit_combo.currentText() == "%":
            # если прооенты то устанавливаем ограничения от 0 доо 100 и два знака после
запятой
            self.comm_val_input.setValidator(QDoubleValidator(0.0, 100.0, 2))
        else:
            # если сумма то от но до миллиарда и тож двумя знаками после запятой
            self.comm_val_input.setValidator(QDoubleValidator(0.0, 99999999.0, 2))

    def update_term_validation(self):
        # для срока кредита ограничения
        self.term_input.setValidator(QIntValidator(1, 999))

```

```

#сброс подсветки
def clear_single_error(self, widget):
    # меняем ошибку на нет, потом забываем что поле было красное и применим стандартный
    # вид полю
    widget.setProperty("error", "false")
    widget.style().unpolish(widget)
    widget.style().polish(widget)

def clean_numeric_input(self, widget):
    # берем текст из поля и убираем пробелы
    raw_text = widget.text().strip().replace(' ', '')
    #заменяем запятую на точку
    clean_text = raw_text.replace(',', '.')
    # если точек больше одной то, склеиваем все до последней точки, которая отделяет две
    # цифры
    if clean_text.count('.') > 1:
        parts = clean_text.split('.')
        return "".join(parts[:-1]) + "." + parts[-1]
    return clean_text

def start_calculation(self):
    # создаем пустые списки для хранения полей и текстов ошибок
    errors_widgets = []
    error_messages = []
    # для объекта дата, изначально пустая
    issue_date_obj = None

    # применяем вот ту очистку поле ввода суммы
    # убираем лишние точки запятые пробелы
    sum_text = self.clean_numeric_input(self.sum_input)
    # для досрочного погашения если галочка стоит
    early_sum_text = ""
    if self.check_early.isChecked():
        early_sum_text = self.clean_numeric_input(self.early_sum_input)

    # для комиссии если она в рублях
    comm_sum_text = ""
    if self.check_comm.isChecked() and self.comm_unit_combo.currentText() == "руб.":
        comm_sum_text = self.clean_numeric_input(self.comm_val_input)

    # проверка суммы, если меньше нуля ошибка
    try:
        if not sum_text or float(sum_text) <= 0:
            errors_widgets.append(self.sum_input)
            error_messages.append("• Сумма кредита (число больше 0)")
    except ValueError:
        # если есть нечисловые символы, то ошибка
        errors_widgets.append(self.sum_input)
        error_messages.append("• Введите корректную сумму")

    #Дата выдачи
    # берем текст без лишник символов
    date_text = self.date_input.text().strip()
    # узнаем сегодняшнее число
    today = datetime.now()
    try:
        # превращаем текст в формат даты, если не в виде ДД.ММ.ГГГГ то будет ошибка
        issue_date_obj = datetime.strptime(date_text, "%d.%m.%Y")
        # кредит не может быть выдан завтра
        if issue_date_obj > today:
            errors_widgets.append(self.date_input)
            error_messages.append("• Дата выдачи не может быть в будущем")
    except ValueError:
        errors_widgets.append(self.date_input)
        error_messages.append("• Введите корректную дату выдачи (ДД.ММ.ГГГГ)")

    #Срок кредита
    term_text = self.term_input.text().strip()
    if not term_text:
        errors_widgets.append(self.term_input)
        error_messages.append("• Введите срок кредита")
    else:
        try:
            # срок всегда целое число
            term_val = int(term_text)
            # проверяем что выбрано лет или мес
            unit = self.term_unit_combo.currentText()

```

```

        # если лет то не больше 50
        if unit == "лет":
            if not (1 <= term_val <= 50):
                errors_widgets.append(self.term_input)
                error_messages.append("• Срок кредита должен быть от 1 до 50 лет")
        else:
            # если месяц то не больше 600
            if not (1 <= term_val <= 600):
                errors_widgets.append(self.term_input)
                error_messages.append("• Срок должен быть от 1 до 600 месяцев")
    except ValueError:
        errors_widgets.append(self.term_input)
        error_messages.append("• Введите число в поле срока")

#Ставка
if self.btn_fixed.isChecked():
    # Получаем очищенный текст из поля фиксированной ставки
    rate_text = self.clean_numeric_input(self.rate_input_fixed)
    if not rate_text:
        errors_widgets.append(self.rate_input_fixed)
        error_messages.append("• Введите процентную ставку")
    else:
        try:
            val = float(rate_text)
            if not (1 <= val <= 100):
                errors_widgets.append(self.rate_input_fixed)
                error_messages.append("• Ставка должна быть от 1% до 100%")
        except ValueError:
            errors_widgets.append(self.rate_input_fixed)
            error_messages.append("• Введите корректное число в поле ставки")
elif self.btn_variable.isChecked():
    # Берем текст, очищаем его от пробелов и меняем запятую на точку
    raw_rate = self.clean_numeric_input(self.rate_input)

    if not raw_rate:
        errors_widgets.append(self.rate_input)
        error_messages.append("• Введите начальную процентную ставку")
    else:
        try:
            rate_val = float(raw_rate)
            if not (1 <= rate_val <= 100):
                errors_widgets.append(self.rate_input)
                error_messages.append("• Ставка должна быть от 1% до 100%")
        except ValueError:
            errors_widgets.append(self.rate_input)
            error_messages.append("• Некорректный формат ставки")

#Вид платежа
# проверяем нажат ли кнопка типа платежа
if not self.btn_annuity.isChecked() and not self.btn_diff.isChecked():
    error_messages.append("• Вид платежа")
# проверяем тип выплат
if not self.btn_interest_only.isChecked() and not self.btn_full_pay.isChecked():
    error_messages.append("• Тип первого платежа")

#День платежа
if self.btn_issue_day.isChecked() or self.btn_last_day.isChecked():
    #Если выбраны кнопки , то убираем ошибку с поля ввода
    self.clear_single_error(self.custom_day_input)
else:
    day_text = self.custom_day_input.text().strip()
    if not day_text:
        errors_widgets.append(self.custom_day_input)
        error_messages.append("• Введите день ежемесячного платежа")
    else:
        try:
            day_val = int(day_text)
            # в месяце не бывает больше 31 дня, добавляем сообщение об ошибке
            if not (1 <= day_val <= 31):
                errors_widgets.append(self.custom_day_input)
                error_messages.append("• День платежа должен быть от 1 до 31")
        except ValueError:
            errors_widgets.append(self.custom_day_input)
            error_messages.append("• Введите число в поле дня платежа")

#Дострочка
if self.check_early.isChecked():

```

```

early_sum_txt = early_sum_text
if not early_sum_txt or float(early_sum_txt or 0) <= 0:
    errors_widgets.append(self.early_sum_input)
    error_messages.append("• Сумма досрочного платежа")

early_date_text = self.early_date_input.text().strip()
try:
    early_date_obj = datetime.strptime(early_date_text, "%d.%m.%Y")
    # нельзя погасить кредит до того как ты его получил
    if issue_date_obj and early_date_obj < issue_date_obj:
        errors_widgets.append(self.early_date_input)
        error_messages.append("• Досрочка не может быть раньше выдачи")
except ValueError:
    errors_widgets.append(self.early_date_input)
    error_messages.append("• Дата досрочного платежа")
# проверка нажатия кнопок по поводу типа досрочного погашения
if not self.btn_reduce_sum.isChecked() and not self.btn_reduce_term.isChecked():
    error_messages.append("• Тип досрочного погашения")
# проверяем кнопку о периодичности погашения
if not self.btn_once.isChecked() and not self.btn_monthly.isChecked() and not
self.btn_yearly.isChecked():
    error_messages.append("• Периодичность досрочного платежа")

#Комиссии
if self.check_comm.isChecked():
    comm_val_txt = self.clean_numeric_input(self.comm_val_input)
    if not comm_val_txt or float(comm_val_txt or 0) <= 0:
        errors_widgets.append(self.comm_val_input)
        error_messages.append("• Значение комиссии")
    # проверка о периодичности
    if not self.btn_comm_once.isChecked() and not self.btn_comm_month.isChecked():
        error_messages.append("• Периодичность комиссии")

#Вывод ошибок
if error_messages:
    # если список не пуст подсвечиваем поля красным
    self.highlight_errors(errors_widgets)
    # создаем всплывающее поле
    msg = QMessageBox(self)
    msg.setIcon(QMessageBox.Icon.Warning)
    msg.setWindowTitle("Ошибка заполнения")
    msg.setText("Для расчета необходимо:")
    # делаем переносы строки в тексте ошибки
    msg.setInformativeText("\n".join(error_messages))
    # показываем окно ошибки
    msg.exec()
    # выходим из функции, чтобы расчет не начался
    return
# если ошибок нет, снимаем всю красную подстветку
self.clear_all_errors()

#Расчет

try:
    # превращаем очищенную строку в дробное число
    sum_credit = float(sum_text)
    term_val = int(self.term_input.text())
    # приводим срок к месяцам
    months = term_val * 12 if self.term_unit_combo.currentText() == "лет" else
term_val

    # День платежа
    if self.btn_issue_day.isChecked():
        # день при выдаче
        payment_day = issue_date_obj.day
    elif self.btn_last_day.isChecked():
        payment_day = 31
    else:
        payment_day = int(self.custom_day_input.text() or 1)

    # данные по ставкам
    rates_history = []
    if self.btn_fixed.isChecked():
        # одна ставка на весь период
        rate_val = self.clean_numeric_input(self.rate_input_fixed)
        rates_history.append((self.date_input.text(), float(rate_val or 0)))
    else:

```

```

# ставка изменяемая
rate_val = self.clean_numeric_input(self.rate_input)
rates_history.append((self.date_input.text(), float(rate_val or 0)))
for i in range(self.rate_changes_layout.count()):
    item = self.rate_changes_layout.itemAt(i)
    if item and item.widget():
        line_edits = item.widget().findChildren(QLineEdit)
        # добавляем в историю дата процент
        if len(line_edits) >= 2 and line_edits[0].text() and
line_edits[1].text():
            clean_rate = self.clean_numeric_input(line_edits[1])
            rates_history.append((line_edits[0].text(), float(clean_rate)))

# создаем словарь с параметрами досрочки
early_params = None
if self.check_early.isChecked() and self.early_sum_input.text():
    early_params = {
        'sum': float(early_sum_text),
        'date': datetime.strptime(self.early_date_input.text(), "%d.%m.%Y"),
        'type': 'reduce_sum' if self.btn_reduce_sum.isChecked() else 'reduce_term'
    }

# создаем словарь с параметрами комиссии
comm_params = None
if self.check_comm.isChecked() and self.comm_val_input.text():
    comm_params = {
        'val': float(self.clean_numeric_input(self.comm_val_input)),
        'unit': self.comm_unit_combo.currentText(),
        'once': self.btn_comm_once.isChecked()
    }

# Вызов расчета
schedule = self.build_payment_schedule(
    sum_credit, rates_history, months,
    'annuity' if self.btn_annuity.isChecked() else 'diff',
    payment_day, self.btn_interest_only.isChecked(), early_params, comm_params
)

# отображение результатов
if schedule:
    # считаем сумму всех выплат из колонки 1
    total_paid = sum(row[1] for row in schedule)
    # если есть разовая комиссия то добавляем ее к общему итогу
    if comm_params and comm_params['once']:
        c_val = comm_params['val'] if comm_params['unit'] == 'руб' else sum_credit
    * (comm_params['val'] / 100)
    total_paid += c_val
    # записываем итоги в правильные поля
    # первый платеж
    self.res_payment.setText(f"{schedule[0][1]:.2f} руб.")
    # всего выплат
    self.res_total.setText(f"{total_paid:.2f} руб.")
    # переплата
    self.res_overpay.setText(f"{max(0, total_paid - sum_credit):.2f} руб.")
    # заполняем таблицу на экране
    self.update_table(schedule)

except Exception as e:
    # если где то что то не вышло, то выводим сообщение об ошибке расчета
    QMessageBox.critical(self, "Ошибка", f"Ошибка расчета: {str(e)}")

def build_payment_schedule(self, sum_credit, rates_history, months, payment_type,
    payment_day, first_only_interest,
    early_params, comm_params):
    # список в который складываются строки таблицы
    schedule = []
    # остаток долга равен всей сумме кредита
    remaining_debt = sum_credit
    # текст из поля даты превращаем в объект от которого будем отсчитывать месяцы
    start_date = datetime.strptime(self.date_input.text(), "%d.%m.%Y")
    # берем самую первую ставку из истории
    current_rate = rates_history[0][1]
    # расчет ежемесячной комиссии
    monthly_comm = 0
    if comm_params and not comm_params['once']:
        # если в рублях то норм, если в процентах то считаем от суммы кредита
        monthly_comm = comm_params['val'] if comm_params['unit'] == 'руб' else sum_credit
    * (
        comm_params['val'] / 100)

```



```

# счетчик месяцев
m = 1
# пока не закончились месяцы и остаток долга больше 1 копейки
while m <= months and remaining_debt > 0.01:
    # считаем дату платежа
    pay_date = start_date + relativedelta(months=m, day=payment_day)

    # Проверка изменения ставки, идем по списку дат когда она должна измениться
    for d_str, r_val in rates_history:
        # если дата платежа наступила или прошла дату изменения то обновляем ставку
        if pay_date >= datetime.strptime(d_str, "%d.%m.%Y"):
            current_rate = r_val
    # месячная процентная ставка в долях
    m_rate = current_rate / (12 * 100)
    # проценты за текущий месяц
    interest = remaining_debt * m_rate

    # Расчет основного платежа
    if payment_type == 'annuity':
        # сколько месяцев осталось платить включая текущий
        m_left = months - m + 1
        # формула расчета аннуитетного платежа
        payment = remaining_debt * (m_rate / (1 - (1 + m_rate) ** -m_left))
    else:
        # если дифф, то
        principal_fixed = remaining_debt / (months - m + 1)
        # полный платеж
        payment = principal_fixed + interest

    # если первый платеж только проценты
    if m == 1 and first_only_interest:
        # не уменьшаем долг
        principal = 0
        # платим только проценты
        payment = interest
    else:
        m_left = months - m + 1
        # Платеж и основной долг
        if payment_type == 'annuity':
            payment = remaining_debt * (m_rate / (1 - (1 + m_rate) ** -m_left))
            principal = payment - interest
        else:
            principal = remaining_debt / m_left
            payment = principal + interest

    # уменьшаем общую задолженность на выплаченный кредит
    remaining_debt -= principal

    # Если досрочка, то вычитаем ее
    total_early_this_month = 0
    if early_params:
        # дата когда начинается досрочка
        e_date = early_params['date']
        is_pay_month = False
        # если разовая досрочка то проверяем совпадение месяца и года
        if self.btn_once.isChecked() and pay_date.month == e_date.month and
pay_date.year == e_date.year:
            is_pay_month = True
        # если ежемесячная платим каждый раз, когда дата платежа больше или равна дате
        # начала досрочки
        elif self.btn_monthly.isChecked() and pay_date >= e_date:
            is_pay_month = True
        # если ежегодная то платим раз в год в тот же месяц
        elif self.btn_yearly.isChecked() and pay_date >= e_date and pay_date.month ==
e_date.month:
            is_pay_month = True

    if is_pay_month:
        # берем сумму досрочки но не больше суммы задолженности
        total_early_this_month = min(early_params['sum'], remaining_debt)
        # вычитаем досрочку из основного долга
        remaining_debt -= total_early_this_month

        # Чтобы остаток не стал -0.00, от микро копеек
        if remaining_debt < 0.01: remaining_debt = 0

    # Итоговый платеж за месяц = обычный платеж + комиссия + досрочка

```

```

total_monthly_record = payment + monthly_comm + total_early_this_month
# собираем данные для одной строки таблицы
schedule.append((
    pay_date.strftime("%d.%m.%Y"),
    round(payment + total_early_this_month + monthly_comm, 2),
    round(principal + total_early_this_month, 2),
    round(interest, 2),
    round(remaining_debt, 2)
))
# переходим к следующему месяцу
m += 1
# если долг выплачен, то выходим из цикла
if remaining_debt <= 0: break

return schedule
# вывод в интерфейс
def update_table(self, schedule):
    # количество строк в таблице по размеру нашего списка
    self.payment_table.setRowCount(len(schedule))
    # строка, данные
    for row_idx, row_data in enumerate(schedule):
        # колонка, конкретное число
        for col_idx, value in enumerate(row_data):
            # форматирование чисел
            if isinstance(value, (int, float)) and col_idx > 0:
                # пробелы между тысячами
                display_text = f"{value:,.2f}".replace(",", " ")
            else:
                display_text = str(value)
            # создание ячейки таблицы
            item = QTableWidgetItem(display_text)
            # выравниваем текст по центру
            item.setTextAlignment(Qt.AlignmentFlag.AlignCenter)
            # вставляем ячейку на определенное место
            self.payment_table.setItem(row_idx, col_idx, item)
# чекбокс досрочки
def handle_early_check(self, state):
    # если галочка снята
    if state == 0:
        # очищаем поле и убираем красную подсветку
        self.early_sum_input.clear()
        self.clear_single_error(self.early_sum_input)
# тоже самое с комиссиями
def handle_comm_check(self, state):
    if state == 0:
        self.comm_val_input.clear()
        self.clear_single_error(self.comm_val_input)
# метод очистки всех результатов расчета на экране
def clear_all_data(self):
    super().clear_all_data()
    # обнуляем метки с результатами
    self.res_payment.setText("0.00 руб.")
    self.res_total.setText("0.00 руб.")
    self.res_overpay.setText("0.00 руб.")

if __name__ == "__main__":
    # основной объект приложения Qt
    app = QApplication(sys.argv)
    # создаем экземпляр класса с интерфейсом
    window = interface_user()
    window.showMaximized()
    # запускаем цикл обработки и корректный выход из программы
    sys.exit(app.exec())

```